

Evals and Safety Engineering: The Crash Pack

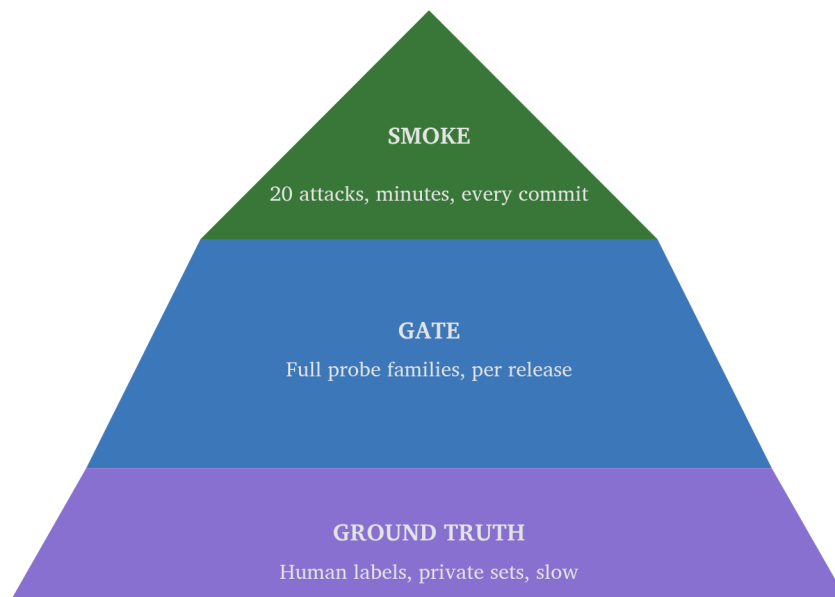
One volume. Zero fluff. Everything in this pack is a job decision, a number you must recall cold, or a trap that burned real teams. It compresses Role Track 5 (eval suite architecture, autorater science, red-teaming, mitigations, measurement, incident response) plus the statistics appendix on non-deterministic evals (11A), the labeler-protocol appendix (11B), and the agent-evals appendix (9A).

How to run this pack

Three passes, about six hours total.

The eval pyramid

Cheap and fast at the top. Expensive and deep at the bottom.



The eval pyramid in one picture. The smoke tier runs 20 known attacks in minutes on every commit. The gate tier runs the full probe families per release. The ground truth tier uses human labels and private sets, slowly. Each tier has one job; never merge them.

How to read this diagram. Read top to bottom, cheap to expensive.

1. **Smoke (top, green):** 20 known attacks, minutes, every commit. What it is: the tripwire. Why it exists: to catch regressions and broken harnesses fast. What breaks if it fails: broken code ships silently.
2. **Gate (middle, blue):** full probe families per release. What it is: the release decision. Why it exists: to measure break rate per family with honest error bars. What breaks if it fails: you ship on vibes.
3. **Ground truth (bottom, purple):** human labels, private sets, slow. What it is: the anchor. Why it exists: to license judges and settle disagreements. What breaks if it fails: every number above it loses its grounding.

- **Pass 1 (90 min):** Read sections 1 and 2. The decision rules and the numbers are the skeleton. Everything else hangs off them.
- **Pass 2 (3 hours):** Read section 3, then do every Q&A in section 4 with the answer hidden. Speak the answer out loud before you reveal it. If you cannot say it, you do not know it.
- **Pass 3 (90 min):** Memorize the cheat sheet in section 5. Test yourself the next morning with a blank page.

This pack teaches judgment, not recipes. The thresholds here (sample sizes, kappa bars, gate limits) serve as starting points from practitioner experience. Your team's real numbers come from your own calibration data. Never ship a threshold you did not earn.

1. Decision rules

Twelve rules. If you remember nothing else, remember these.

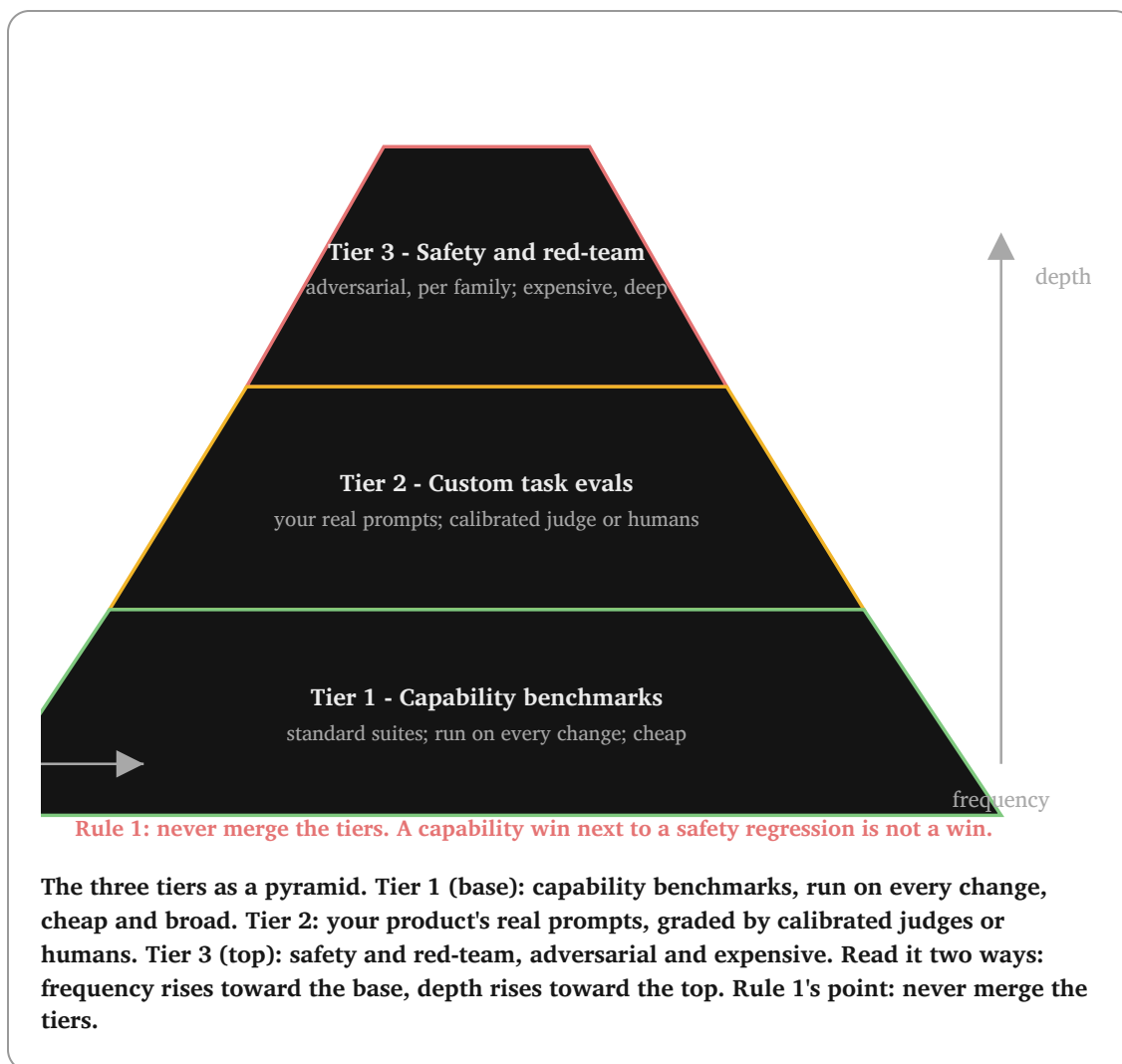
The four parts of an eval suite. Tasks are what you ask the model to do. Graders score the answers: from checkable string checks to human raters to LLM judges. The harness is the code that runs everything: sampling, retries, logging, scoring. The gating policy is the rule that turns scores into a ship or no-ship decision.

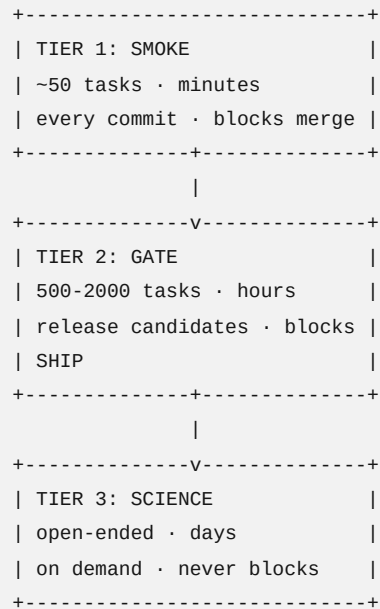
Treat an LLM judge as a measurement instrument, not an oracle. License it like one. That means proving it agrees with humans before you trust a single number from it. Calibration is that proof: comparing the judge against humans on a shared item set until the agreement is good enough to ship.

Rule 1. Split the suite into three tiers, and never merge them

- **Tier 1, smoke:** about 50 tasks, runs in minutes, on every commit. Job: catch obvious breakage fast. A smoke failure blocks the *merge*, not the release.
- **Tier 2, gate:** 500 to 2,000 tasks, runs in hours, on every release candidate. Job: the ship-or-hold decision. Write thresholds down and version them.
- **Tier 3, science:** open-ended, runs for days, on demand. Job: answer research questions ("did the new data mix change refusal quality?"). It lives outside the gate so slow questions never block shipping.

The common failure is one giant suite trying to do all three jobs. It is too slow for commits, too noisy for gating, and too rigid for research. Split the tiers and each one gets simpler.



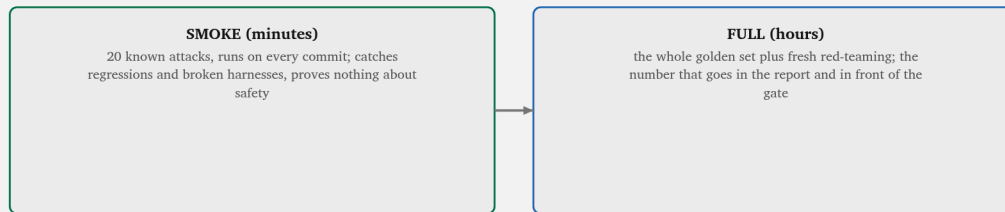


How to read this diagram

1. Time flows downward; strictness flows upward. The cheapest tier runs most often.
2. Smoke blocks the merge, not the release: it answers "is this obviously broken," not "is this shippable."
3. Only Tier 2 carries a ship-or-hold verdict. If Tier 3 questions block shipping, someone merged the tiers and the suite will get routed around.

Smoke vs full eval: fast signal, then the real measurement

smoke tells you in minutes; the full eval tells you the truth



The failure mode is running only the smoke eval and calling it a safety case. Smoke is a tripwire, not a verdict.

Rule: smoke on every commit, full eval on every release candidate, fresh red-team items every quarter.

The three-tier habit in one picture. The smoke eval runs 20 known attacks in minutes on every commit and catches regressions and broken harnesses. The full eval runs the whole golden set plus fresh red-teaming and produces the number that goes in the report. Smoke is a tripwire, not a verdict.

What to look at first: the two boxes side by side, minutes versus hours. The arrow shows the escalation order. The one takeaway: the failure mode is to run only the smoke eval and call it a safety case.

Harness anatomy

Five parts. Miss one and the suite is a pile of notebooks.



Harness anatomy. Seed everything so reruns give the same samples. Isolate each task so one crashing grader never kills the run. Store results as append-only JSONL so old runs stay re-analyzable. Version the golden set like code, and gate on deltas against the baseline, not on absolute scores. Miss one part and the suite is a pile of notebooks.

How to read this diagram. Five numbered parts, each independent.

- 1. Seed everything.** What it is: random seeds, set once, for sampling. Why it exists: so reruns give identical samples and deltas mean something. What breaks if it fails: you cannot tell a regression from sampling luck.
- 2. Isolate each task.** What it is: per-task try/except. Why it exists: so one crashing grader never kills the run. What breaks if it fails: you lose every result after the crash.
- 3. Append-only JSONL.** What it is: one row per result, never overwritten. Why it exists: so old runs stay re-analyzable when graders change. What breaks if it fails: a grader fix forces you to re-run history you cannot reproduce.
- 4. Version the golden set.** What it is: changelog plus retired stale items. Why it exists: because attacks evolve and the set rots. What breaks if it fails: pass rates creep up while real breaks do not fall.
- 5. Gate on deltas.** What it is: candidate versus baseline on the same suite. Why it exists: absolute scores hide regressions. What breaks if it fails: a worse model ships because its score looked fine.

Rule 2. Climb the grader ladder: checkable first, judgment last

Pick the cheapest grader that is honest for the task.

Grader ladder: cheapest honest grader first.

```

+-----+
| deterministic string check | checkable answers only; normalize case and
whitespace
+-----+
      v
+-----+
| human rubric grading      | two humans per item; the entry ticket
+-----+
      v
+-----+
| autorater (LLM judge)    | only after it clears calibration (Rule 4)
+-----+

```

Rule: keep human-graded tasks in the gate tier permanently.

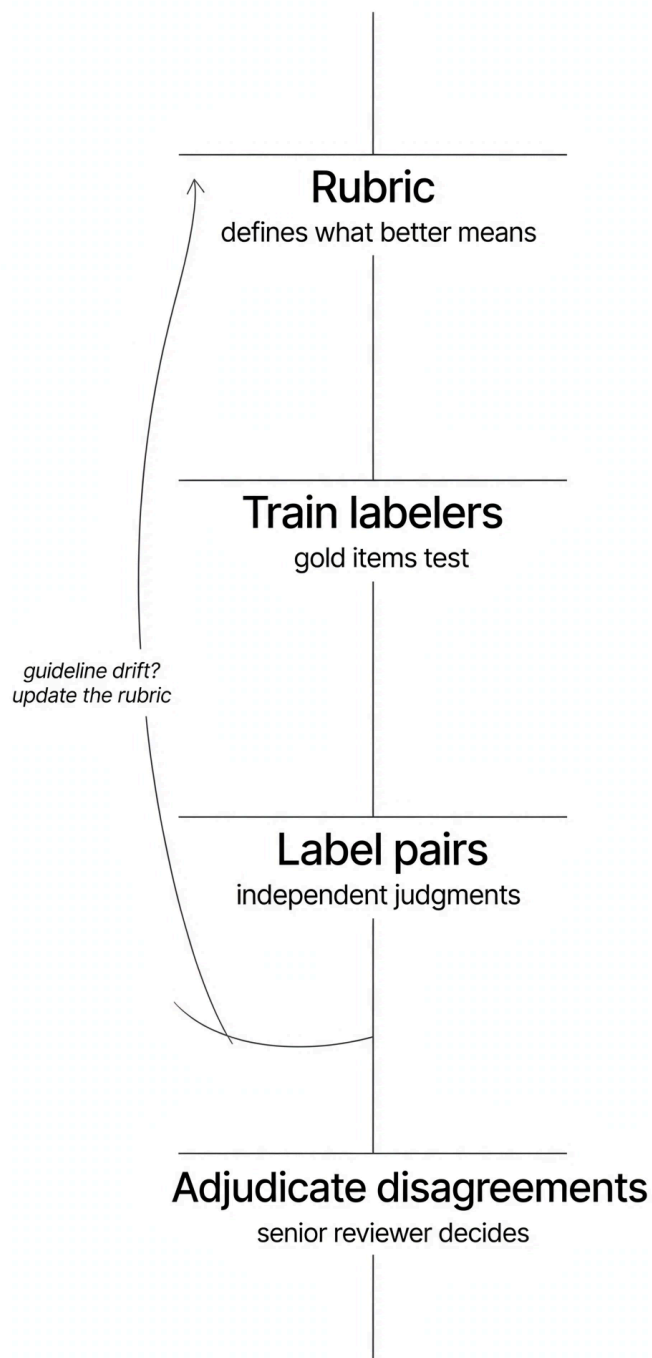
A gate with zero human eyes is a gate nobody trusts when it matters.

1. **Checkable answer?** Use a deterministic string check (normalize case and whitespace first; raw substring matching fails silently on formatting).
2. **Open-ended?** Write a rubric, then grade with humans first. (A rubric is a written list of what earns what score: the grading instructions two humans must be able to apply consistently.) Two humans per item is the entry ticket.
3. **Scale needed?** Add an autorater only after it clears calibration (Rule 4). Keep some human-graded tasks in the gate tier permanently. A gate with zero human eyes is a gate nobody trusts when it matters.

A rubric that two humans cannot apply consistently is not a rubric. Fix it before any autorater sees it.

Rule 3. Never trust a judge you have not licensed

An LLM judge is a measurement instrument. License it like one:



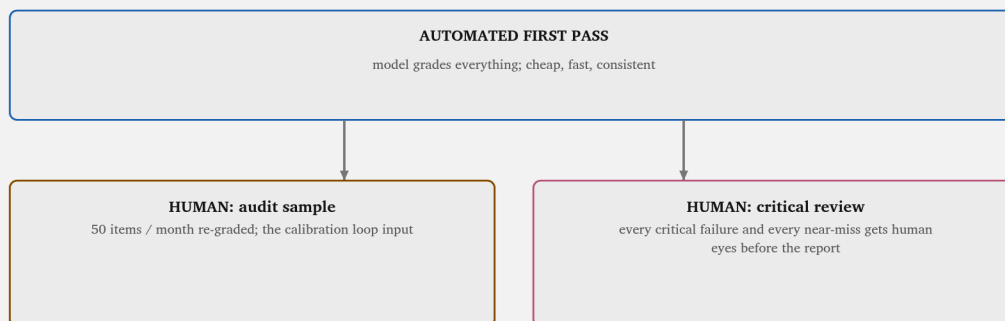
A rubric defines what 'better' means; labelers train against gold items; pairs of labelers judge independently; a senior reviewer adjudicates disagreements. The feedback arrow is the load-bearing part: guideline drift goes back into the rubric, not into silent relabeling. A rubric two humans cannot apply consistently is not a rubric. Figure: curriculum eval-stats appendix (v11).

1. Build a calibration set of 20 to 50 items spanning the full quality range. Deliberately include known-bad and known-borderline cases.
2. Humans label first, 2 to 3 raters per item. (A labeler is a human who scores items.) If humans disagree, the rubric fails. Fix the rubric.
3. Run the judge on the same items. Compute Cohen's kappa (agreement corrected for chance; 2.1 works the formula), not raw agreement.
4. Read the disagreements. Sort by gap size, read the top 10, and ask for each: did the judge miss, did the human miss, or is the rubric ambiguous? The answer is almost always the rubric.
5. **Revise the rubric, not the judge.** Editing the judge prompt to chase individual disagreements overfits with extra steps.
6. Re-run on a fresh sample. Never re-tune on the same golden set twice (the golden set: the fixed, human-scored items every judge licenses itself against) without a fresh holdout.

The bar: judge-vs-human kappa near the human-vs-human ceiling (about 0.8 on many real tasks). A judge far below the ceiling is not a human substitute.

Human review: where the humans sit in the loop

automated grading scales; humans adjudicate what matters



Humans do not grade 10,000 items; they grade the 50 that keep the machine honest and the handful that could end a career.

Automated grading scales to everything; humans adjudicate what matters. Humans re-grade fifty items a month as the calibration loop input. Every critical failure and near-miss gets human eyes before anyone writes the report.

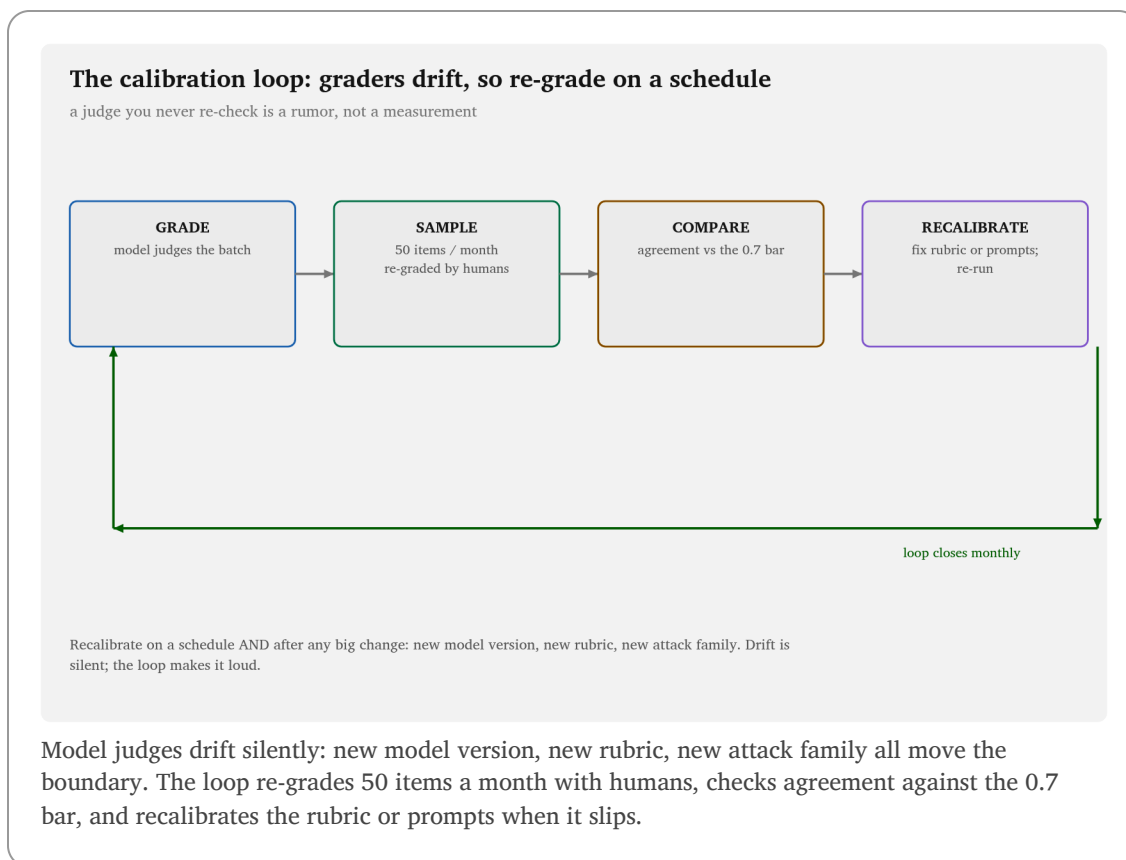
What to look at first: the wide automated box on top, then the two human boxes it fans out to. The one takeaway: humans do not grade 10,000 items. They grade the 50 that keep the machine honest and the handful that could end a career.

Rule 4. Calibrate on a schedule, and after every big change

A calibrated judge goes stale, so re-calibrate on a calendar schedule and immediately after any big model change. Between calibrations, keep the judge on a dashboard with three panels:

- Agreement with humans on a rolling sample (re-grade 50 items a month).
- Score distribution over time (a sudden shift means something changed).
- Monthly disagreement review.

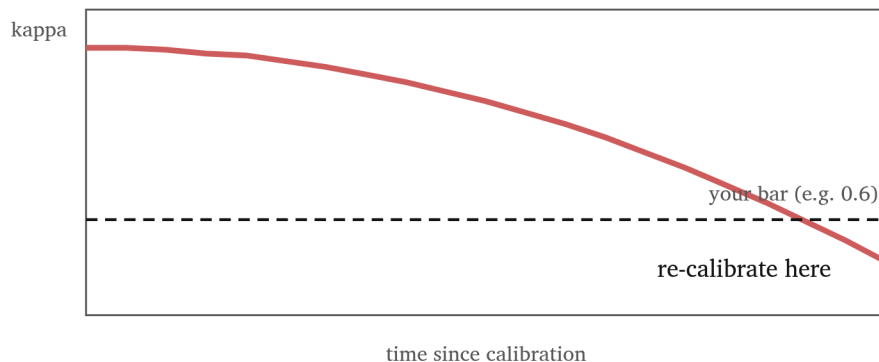
When kappa drops below your bar, the judge goes back through the calibration loop. A judge that is not monitored is a judge you are not using.



What to look at first: the four boxes left to right, then the green arrow that closes the loop back to grading. The one takeaway: a judge you never re-check is a rumor, not a measurement.

Judge staleness: kappa decays

A calibrated judge goes stale. Re-calibrate on a schedule.



Distribution shift, prompt leakage, and gaming all push kappa down.

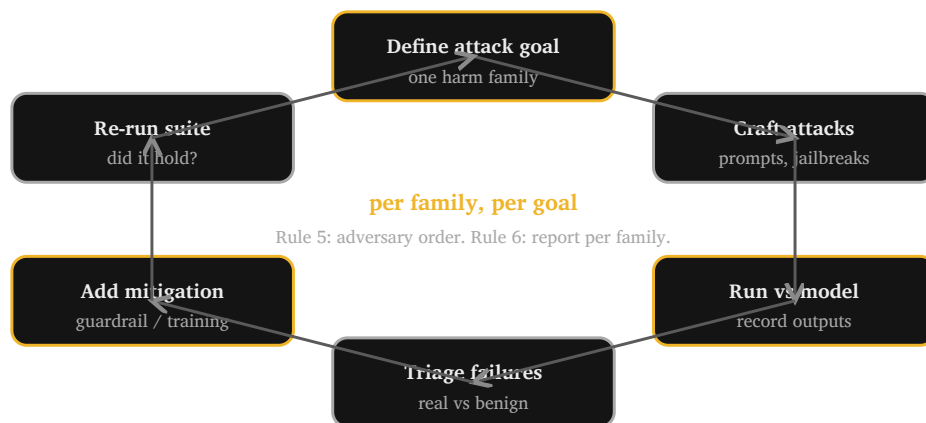
Judge staleness. A freshly calibrated judge starts near its licensed kappa. Distribution shift, prompt leakage, and gaming push kappa down over time. When kappa drops below your bar, re-calibrate. A judge you never re-check is a rumor, not a measurement.

How to read this diagram. Time runs left to right; kappa runs bottom to top.

1. **The red curve** is kappa over time since calibration. It falls because the world moves under the judge: new model styles, new slang, new attacks.
2. **The dashed line** is your bar (for example 0.6). It is the minimum agreement you accept.
3. **The crossing point** is the re-calibration trigger. When the curve crosses the line, the judge goes back through the calibration loop.

Rule 5. Red-team in adversary order, goal by goal

Test against the three adversary profiles in order: the curious user first, then the motivated amateur, then the skilled adversary. If the curious user breaks your model, you do not have a skilled-adversary problem. You have a basics problem.



The library of attacks stays alive: every incident adds a case (Rule 12). A red-team that reuses last quarter's prompts is a

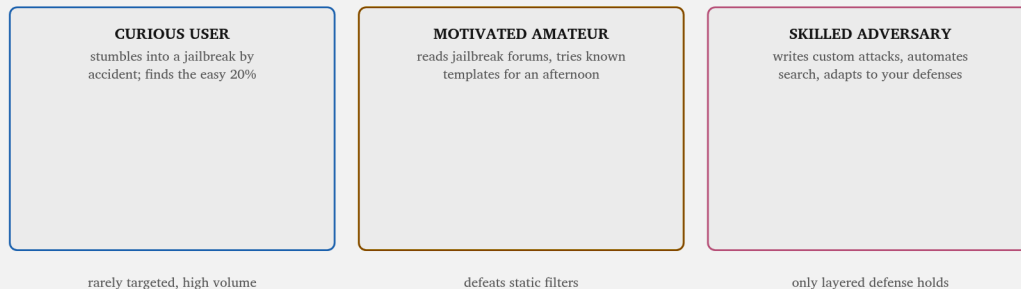
The red-team loop, per family, per goal. Define one attack goal. Craft attacks. Run them against the model. Triage which failures are real. Add the mitigation. Then re-run the suite to check it held. Rule 6's point: the attack library stays alive; every incident adds a case. A red-team that reuses last quarter's prompts is a compliance ritual.

Cover all four attacker goals with separate probe sets. A model can be strong against one and weak against another. The goals: refusal bypasses, prompt extraction, injection (instructions hidden in tool output or pasted content), (A jailbreak is a prompt crafted to make the model bypass its refusal training. Prompt extraction gets the model to reveal its system prompt or hidden instructions.) misuse enablement.

Cover all six attack families for every high-risk goal: direct requests, roleplay and persona, encoding and translation, multi-turn buildup, instruction smuggling, automated search (static libraries and adaptive orchestrators). If a direct request works, stop. Fix the basics before you hunt for clever attacks.

Three adversary profiles: the threat model behind every red-team plan

same model, three very different attackers



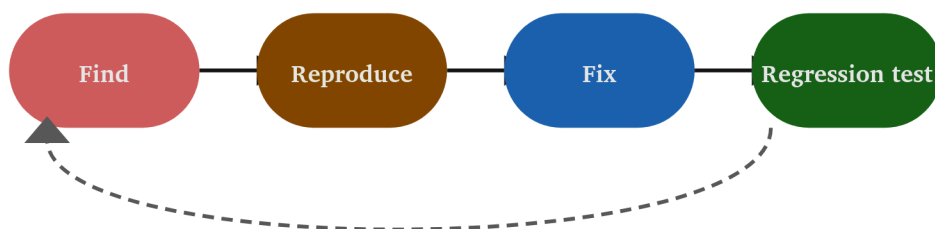
Your eval is only as strong as the profile it assumes. A defense that stops the curious user says nothing about the skilled adversary.

The same model faces three very different attackers. The curious user stumbles into the easy 20%. The motivated amateur tries known templates for an afternoon. The skilled adversary writes custom attacks and automates the search. Your eval is only as strong as the profile it assumes.

What to look at first: the three boxes left to right, escalating. The one takeaway: a defense that stops the curious user says nothing about the skilled adversary.

Red-team loop

Every finding becomes a permanent test.



The loop never ends: new attacks feed the next round.

The red-team loop. Find a break. Reproduce it with exact prompts, model version, and settings. Fix it. Turn the reproduction into a regression test in the suite. The loop never ends: new attacks feed the next round, and every incident ends in the eval suite.

How to read this diagram. Four stages left to right, then the dashed arrow loops back.

1. **Find.** What it is: a break discovered by probing. Why it exists: to surface what the suite missed. What breaks if it fails: unknown holes stay unknown.
2. **Reproduce.** What it is: exact prompts, model version, settings. Why it exists: a finding nobody can reproduce is a rumor. What breaks if it fails: engineers cannot fix what they cannot replay.
3. **Fix.** What it is: the mitigation. Why it exists: to close the hole. What breaks if it fails: the break ships to users.
4. **Regression test.** What it is: the reproduction becomes a permanent suite item. Why it exists: so the hole never reopens silently. What breaks if it fails: the same break returns in a later release.

Rule 6. Report red-team results per family, and treat the library as living

One number ("12% of probes broke it") tells you nothing. Report break rates per attack family: the family view shows which *kinds* of attacks are weak. When a new jailbreak technique appears in public, add it to the library as a new probe. A finding is reproducible or it is not a finding: every reported break ships with the exact prompts, model version, and settings that reproduce it.

The six attack families: one ladder, one goal each

all adversarial pressure on a model is some form of these six

1. direct

ask outright; the baseline everything else is measured against

2. roleplay / persona

"you are a helpful security researcher" and its cousins

3. encoding / translation

base64, leetspeak, low-resource languages hide the request

4. multi-turn

harmless steps that compose into a harmful whole

5. instruction smuggling

system-style directives hidden inside user content

6. automated search

optimizers that learn the jailbreak for you

Memorize the six names cold. In an eval report, naming the family the break belongs to is what turns an anecdote into coverage.

All adversarial pressure on a model is some form of these six. Memorize the names cold: in an eval report, naming the family a break belongs to is what turns an anecdote into coverage.

What to look at first: the six rows top to bottom. The one takeaway: file every finding under its family the week it arrives, and the quarterly report writes itself.

Rule 7. Stack mitigations in order, and ask each layer what it misses

Read bottom to top. Each layer catches what the layer below missed.

```

      USER
      |
      v
+-----+
| 5. MONITORING ..... log, sample, alert on      |
|   production traffic; catch what shipped anyway |
+-----+
| 4. OUTPUT FILTER .. classifier on the response  |
|   before it reaches the user                  |
+-----+
| 3. THE MODEL ..... refusal training, values,   |
|   instruction hierarchy from post-training     |
+-----+
| 2. INPUT FILTER ... classifier on the prompt   |
|   before it reaches the model                 |
+-----+
| 1. SYSTEM PROMPT .. hardened instructions,    |
|   least privilege, secret minimization        |
+-----+

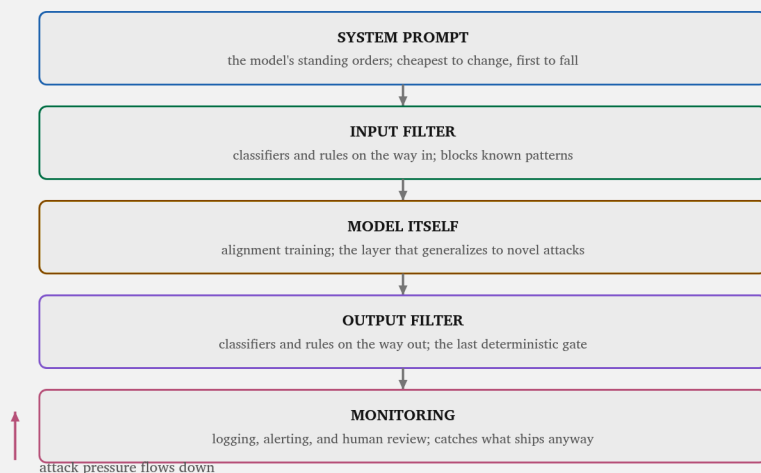
```

Two terms from the diagram. Post-training is the training phase after pre-training that teaches refusal and values. The instruction hierarchy is the rule that instructions from higher-priority sources outrank lower ones. System prompt first, then the developer, then the user, then tool output.

For each layer ask two questions: what does this layer catch that the others miss, and what sails straight through it? System-prompt hardening is cheap and fast to deploy, and weaker than training. Use it as a layer, never as the plan. A filter with unknown precision and recall is not a mitigation; it is a hope.

The mitigation stack: five layers, each catching what the last missed

no single layer survives the skilled adversary; the stack might



Defense in depth is not a slogan: measure the break rate with each layer on and off, so you know which layer is actually load-bearing.

Five layers, each catching what the last missed. The system prompt is cheapest to change and first to fall. The model itself is the layer that generalizes to novel attacks. Monitoring catches what ships anyway. No single layer survives the skilled adversary; the stack might.

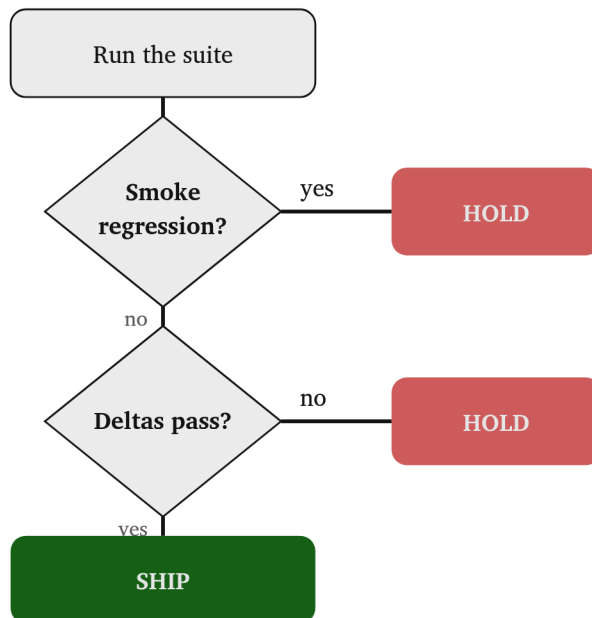
What to look at first: the five layers top to bottom in processing order, with attack pressure flowing down the red arrow on the left. The one takeaway: measure the break rate with each layer on and off, so you know which layer is actually load-bearing.

Rule 8. Gate on deltas against the baseline, not on absolute scores

A 92% that used to be 95% ships under an absolute bar. Deltas catch decay. (A delta is the change from the released baseline.) Every release diffs the candidate against the released baseline on the same suite, same seeds. Version the gate policy as code, not as a number in someone's head:

The gate: ship or hold

Every release walks this path.



The gate decision. Run the suite. If any smoke test regresses, hold. If the deltas against the baseline fail, hold. Otherwise ship. The gate runs on deltas, not on absolute scores, and a human confirms every hold.

How to read this diagram. Follow the arrows top to bottom.

1. **Run the suite.** Same suite, same seeds as the baseline.
2. **Smoke regression?** Any smoke regression holds the release. Smoke is the tripwire; a trip means stop.
3. **Deltas pass?** The candidate must beat or match the baseline within the policy. If not, hold.
4. **Ship.** Both checks green means the release goes. A human still confirms every hold before it becomes final.

Gate on the delta, not the absolute score

baseline v3



candidate



intervals overlap: inconclusive. Add eval items (Rule:

baseline v3



candidate v4



clean separation: ship it. Report the delta with its int

How to read: each bar is a model's score with its 95% interval. Top pair: the intervals overlap, so the 'win' is noise; add eval items until the interval shrinks, do not ship on vibes. Bottom pair: clean separation, so the delta is real. Report the delta with its interval, never a single number.

- **max smoke regressions: 0.** Any smoke regression holds.
- **max probe break rate: 10%** in any attack family.
- **judge kappa below 0.6: warn.** The gate flags; a human confirms before a HOLD becomes final, because automated scoring has false positives.
- **Human confirms every HOLD.** The gate proposes; a person disposes.

The sign-off reads: "Model v3.2 passes the gate suite at thresholds T, with N exceptions listed below." List the exceptions with owners and dates. That sentence separates engineering from theater.

Rule 9. Set filter thresholds per category, by risk

Every filter lives on a precision-recall tradeoff (precision: of the flagged items, how many were really bad; recall: of the really bad items, how many the filter caught), and the threshold is a product decision, not a math fact. Set it strict for self-harm and weapons content, more balanced for spam. Then measure the filter like any component. Measure precision and recall on a labeled set, latency added to the serving path, and the false-positive rate on real traffic. Filters need adversarial testing of their own: paraphrase, obfuscation, and splitting content across messages are the standard evasions.

Rule 10. When the public suite and the private set disagree, believe the private set

Contamination defense in five moves:

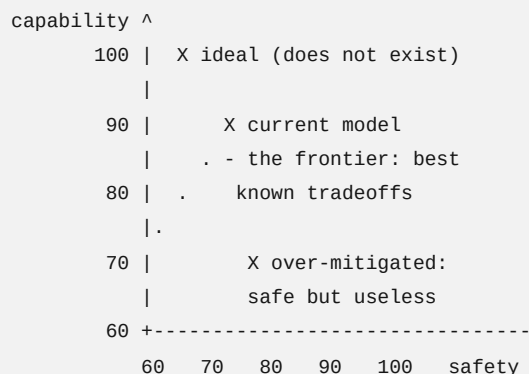
1. Version every eval like code.
2. Keep a changelog (when you add, remove, or reword a task, log why).
3. Keep eval sets private.

4. Rotate items and retire saturated tasks.
5. Hold out a private slice that nobody trains against and nobody tunes prompts against.

Watch for suspicious jumps on static sets. A suite where everything scores 99% measures nothing.

Rule 11. Report capability and safety side by side, always

Safety costs capability: a model that refuses more also refuses some benign requests, and a strict output filter blocks some good traffic. This is the shape of the problem, not a flaw in your mitigations. The honest picture is a Pareto frontier: the best safety at each level of capability. Pick a point on it deliberately and say which point you picked. The unit of honest safety engineering is one sentence. Example: "Refusal tuning cut harmful compliance from 4% to 0.5% and cost 1.2 points on benign task completion."



How to read this diagram

1. The top-right corner (perfect at both) does not exist. Every real model is a point on or below the curve.
2. Moving right (safer) usually moves the point down (less capable). The curve is the set of best-known tradeoffs.
3. A release moves the point. Your job is to say where it moved and what it cost, with both numbers side by side.

The eval report card: one page that says what you tested and what broke

if it is not in the report, it did not happen

SCOPE

model version, prompt version, dates, who ran it

GOLDEN SET

version, size, coverage per failure bucket

RESULTS

pass rate, break rate per family, kappa of the graders

CRITICAL FAILURES

each one: transcript, bucket, severity, owner, fix date

LIMITATIONS

what the eval does NOT cover; the next scheduled run

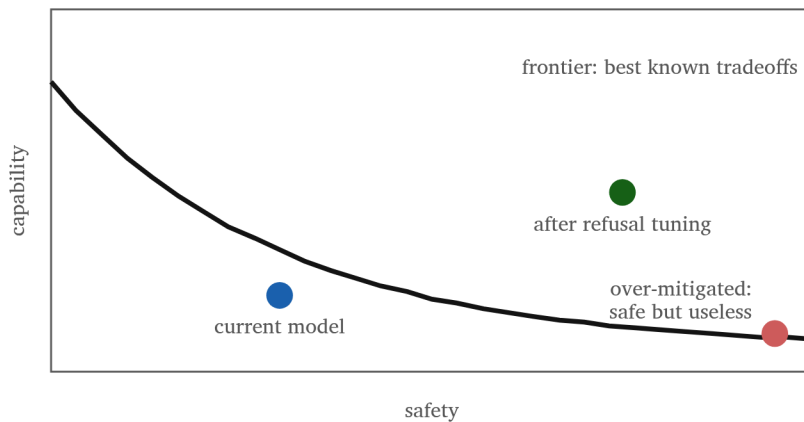
The report card is the product. A perfect eval with no readable report changes nothing; a plain report that names the breaks gets them fixed.

One page that says what you tested and what broke. It lists model and prompt versions, the golden set version and its per-bucket coverage, and pass and break rates with grader kappa. It lists every critical failure with transcript and owner, and what the eval does not cover. If it is not in the report, it did not happen.

What to look at first: the five rows top to bottom. The one takeaway: the report card is the product; a perfect eval with no readable report changes nothing.

Pareto frontier: safety vs capability

Every release moves the point. Say where it moved and what it cost.



The top-right corner (perfect at both) does not exist.

The Pareto frontier. Each point is a model: how safe versus how capable. The curve is the best known tradeoffs, and the top-right corner (perfect at both) does not exist. Refusal tuning moved the point up and left: safer, slightly less capable. Over-mitigation lands bottom-right: safe but useless. Pick a point deliberately and say which point you picked.

How to read this diagram. Safety runs left to right; capability runs bottom to top.

1. **The curve** is the Pareto frontier: the best safety at each level of capability. No model sits above it.
2. **Blue point:** the current model. **Green point:** after refusal tuning: safer, a little less capable.
3. **Red point:** over-mitigated: very safe, too useless to ship. Safety without capability is a brick.
4. **The empty top-right** is the point that does not exist: perfect at both. Every release moves the point; your job is to say where it moved and what it cost, with both numbers side by side.

Rule 12. Every incident ends in the eval suite

Triage with a severity scale written for model behavior. S1 is active, ongoing harm at scale (page now). S2 is serious but bounded (same-day response, mitigation within the week). S3: concerning but rare (normal cycle). S4 is a paper cut (log it).

Incident severity: S1 to S4

Severity sets the clock. The clock sets the response.

S1	Active harm at scale Response: Page now
S2	Harm likely, not yet widespread Response: Same day
S3	Potential harm, contained Response: Normal cycle
S4	Low risk, log and watch Response: Log it

Incident severity. S1 means active harm at scale: page now. S2 means harm is likely but not yet widespread: same-day response. S3 means concerning but contained: normal cycle. S4 is a paper cut: log it. Severity sets the clock, and the clock sets the response.

How to read this diagram. Four rows, worst at the top.

1. **S1 (red):** active, ongoing harm at scale. Page the on-call lead and the safety lead now.
2. **S2 (amber):** serious but bounded. Same-day response, mitigation within the week.
3. **S3 (blue):** concerning but rare. Normal engineering cycle.
4. **S4 (gray):** a paper cut. Log it and move on.

The first hour decides the severity. Reproduce, assess blast radius, decide hold versus rollback, assign one owner.

The first hour has a checklist:

1. Reproduce the behavior and record exact prompts, model version, and settings.
2. Assess blast radius and check whether people share it publicly. (Blast radius: how far the harm reaches: who it affects and how widely it spreads.)
3. Decide hold vs rollback.
4. Open the incident record.
5. Assign one owner.

Write rollback criteria before the incident, in the units you already measure, and rehearse the rollback path. A rollback you have never rehearsed is a plan you do not have. Record near-misses: an S2 that almost became an S1 is free information about where your criteria are thin.

Postmortems are blameless and ask five questions in order:

1. What happened (timeline of facts).
2. What was the impact (numbers).
3. Why did it happen (five whys).
4. What did we learn about our process.
5. What changes now (action items with owners and dates).

The most important rule: the incident's reproduction prompts become regression tasks in the suite. Build the missing probe family. Give the metric that should have caught it a threshold. An incident that does not change the eval suite will happen again.

Next: Section 2, the numbers behind the rules: kappa, Wilson intervals, pass@k, majority-vote math, and the constant sheet.

The eval contract: what "we tested it" has to mean

no contract, no claim; a vibes-based eval is a press release

GOLDEN SET

100-300 items, fixed version, covers all four failure buckets

DETERMINISTIC GRADER

same input, same score, every run; temperature 0, pinned version

PASS THRESHOLD

a number plus a no-critical-failure rule; "mostly fine" is not a threshold

FAILURE TAXONOMY

every miss filed under refusal / extraction / injection / misuse

RE-RUN RULE

when the eval re-runs: schedule plus after every model or prompt change

Write the contract before the model exists. An eval designed after the results are in is a rationalization with a spreadsheet.

What "we tested it" has to mean before anyone allows the claim. A versioned golden set covering all four failure buckets. A grader that scores the same input the same way every run. A numeric pass threshold plus a no-critical-failure rule. A re-run rule tied to a schedule and to every model or prompt change.

What to look at first: the five contract clauses top to bottom. The one takeaway: write the contract before the model exists; an eval designed after the results are in is a rationalization with a spreadsheet.

2. Numbers and formulas to memorize

The rules say what to build. Section 2 gives you the arithmetic to trust the build: agreement metrics, sample-size math, and the pass@k family. Then you can tell a real improvement from noise.

Every snippet below ran on this machine. The printed outputs are real.

2.1 Cohen's kappa: the agreement number that does not flatter

$$\kappa = (p_o - p_e) / (1 - p_e)$$

Where: p_o means observed agreement. p_e means the agreement the two raters' individual yes-rates produce by chance.

Landis and Koch bands:

- Below 0: worse than chance.
- 0.00-0.20: slight.
- 0.21-0.40: fair.
- 0.41-0.60: moderate.
- 0.61-0.80: substantial.
- 0.81-1.00: almost perfect.

Worked: `cohen_kappa(60, 10, 15, 15) = 0.375`, a fair rating. The lazy judge (always says pass) gets $\kappa = 0.000$: raw agreement looks fine, chance-corrected agreement is zero.

Landis and Koch bands:

- Below 0: worse than chance.
- 0.00-0.20: slight.
- 0.21-0.40: fair.
- 0.41-0.60: moderate.
- 0.61-0.80: substantial.
- 0.81-1.00: almost perfect.

Agreement is not accuracy: correct for chance

judge vs human, 100 items

	judge says good	judge says bad
human says good	60	10
human says bad	15	15

Raw agreement: $75 / 100 = 75\%$

Looks fine. But both sides lean toward "good":

humans say good 75%, the judge says good 70%.

Chance alone would agree 60% of the time.

$$\text{kappa} = (0.75 - 0.60) / (1 - 0.60) = 0.375$$

Agreement beyond chance, on a -1 to 1 scale.

0.375 is only fair: the judge is mostly riding the base rate.

License bar used in this pack: 0.8.

Kappa punishes easy wins. That is why it beats raw agreement.

Why kappa beats raw agreement. 75% of the items match, but both sides lean toward "yes": chance alone would agree 60% of the time. Kappa strips that out: $(0.75 - 0.60) / (1 - 0.60) = 0.375$, a fair rating on the Landis-Koch bands, and well below this pack's 0.8 licensing bar.

```

def cohen_kappa(a, b, c, d):
    # Confusion matrix, judge rows vs human columns:
    #   a = both say yes   b = judge yes, human no
    #   c = judge no, human yes   d = both say no
    # Using the matrix is the whole point: kappa corrects for the agreement
    # the two yes-rates would produce by luck. Raw percent agreement skips
    # that correction and flatters lazy judges.
    n = a + b + c + d
    if n == 0:
        raise ValueError("empty confusion matrix: nothing to compare")
    p_o = (a + d) / n          # Observed agreement: the diagonal over total.
    p_yes_judge = (a + b) / n  # Judge's individual yes-rate.
    p_yes_human = (a + c) / n  # Human's individual yes-rate.
    # Chance agreement: yes-yes by luck plus no-no by luck.
    p_e = p_yes_judge * p_yes_human + (1 - p_yes_judge) * (1 - p_yes_human)
    if p_e == 1.0:
        return 1.0 # Degenerate: both raters said the same thing every time.
    return (p_o - p_e) / (1 - p_e)

def kappa_band(k):
    # Band edges are the published Landis-Koch cutoffs.
    if k < 0.0: return "worse than chance"
    if k <= 0.20: return "slight"
    if k <= 0.40: return "fair"
    if k <= 0.60: return "moderate"
    if k <= 0.80: return "substantial"
    return "almost perfect"

# Worked example: judge passes 60, human passes 10, they agree on 15 of 100.
k = cohen_kappa(60, 10, 15, 15)
print(f"kappa = {k:.3f}")          # kappa = 0.375
print("band:", kappa_band(k))     # band: fair

# The lazy-judge check: always says pass. Raw agreement looks fine.
k2 = cohen_kappa(70, 0, 30, 0)
print(f"lazy judge kappa = {k2:.3f}") # lazy judge kappa = 0.000

```

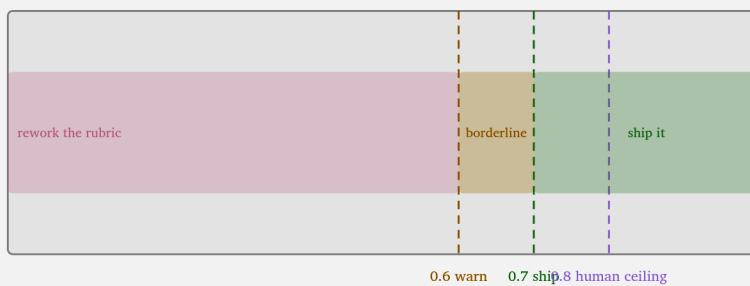
How to read this diagram

1. The function takes the four cells of the confusion matrix (the 2x2 table: human-yes/judge-yes, human-yes/judge-no, human-no/judge-yes, human-no/judge-no). Those four numbers carry both the agreement and each rater's individual yes-rate.
2. p_o is plain observed agreement. p_e is the chance term raw agreement pretends does not exist. A judge with a 70% yes-rate and a human with a 75% yes-rate agree on "yes" 52.5% of the time by guess.
3. The worked example prints 0.375: raw agreement said 75%, kappa says "fair." The 37.5-point gap is chance agreement the raw number hid.
4. The lazy judge gets kappa exactly 0.0. Kappa exposes what raw agreement rewards.

Memorize: **raw agreement flatters; kappa corrects**. The human ceiling is about kappa 0.8 on many real tasks (Rule 3's bar), so judge-vs-human against a perfect 1.0 is an unfair bar. Report both numbers side by side.

Inter-rater agreement: kappa turns "we agree" into a number

two graders, one rubric, one statistic



Kappa below 0.6 means the graders disagree enough that the labels are noise: fix the rubric, not the graders. At or above 0.7 the labels are trustworthy. Human ceiling sits near 0.8, so do not chase 0.95; you will never get there.

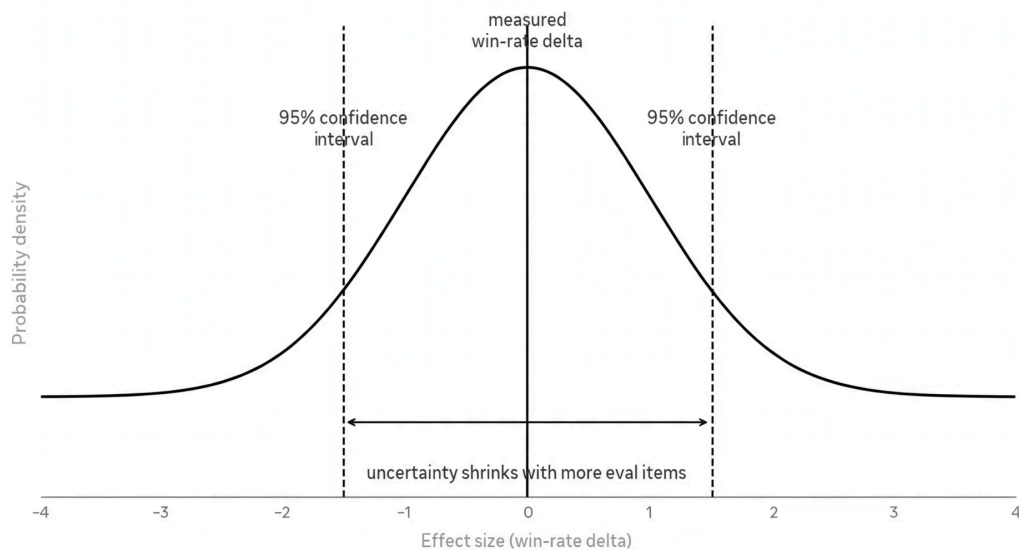
Rule of thumb: $\text{kappa} \geq 0.7$ ships, $\text{kappa} < 0.6$ sends the batch back for rubric repair.

Kappa turns "we agree" into a number that does not flatter. Below 0.6 the graders disagree enough that the labels are noise: fix the rubric, not the graders. At or above 0.7 the labels are trustworthy. The human ceiling sits near 0.8, so do not chase 0.95.

What to look at first: the three dashed thresholds and the colored zones. The one takeaway: kappa below 0.6 sends the batch back for rubric repair.

2.2 Wilson interval: honest error bars for small, lopsided samples

For a yes/no rate with n trials and $z = 1.96$ (95%):



The interval is the honest statement: the truth could be anywhere inside it. More eval items shrink the interval. This is the picture behind the Wilson formula in this section. Small, lopsided samples get wide intervals. Wide intervals mean you do not know yet. Figure: curriculum eval-stats appendix (v11).

$$\begin{aligned}\text{center} &= (\hat{p} + z^2/2n) / (1 + z^2/n) \\ \text{half-width} &= z \cdot \sqrt{(\hat{p}(1 - \hat{p})/n + z^2/4n^2) / (1 + z^2/n)} \\ \text{interval} &= [\text{center} - \text{half}, \text{center} + \text{half}]\end{aligned}$$

Where: $\hat{p}$ is the observed rate, n is the trial count, $z = 1.96$ for a 95% interval. Use Wilson instead of the plain bootstrap when the sample is small and lopsided. A bootstrap of 20 hits out of 20 tries claims a width of zero, which is a lie.

Worked: `wilson_interval(20, 20)` gives $20/20 \rightarrow$ 95% CI [0.839, 1.000]. Twenty successes buy you a lower bound of 0.839, not 1.0.

Use Wilson instead of the bootstrap when the sample is small and lopsided. (The bootstrap: resampling your data with replacement to estimate uncertainty.) Take 20 hits out of 20 tries. A plain bootstrap claims a confidence interval of [1, 1], which is certainty you do not have.

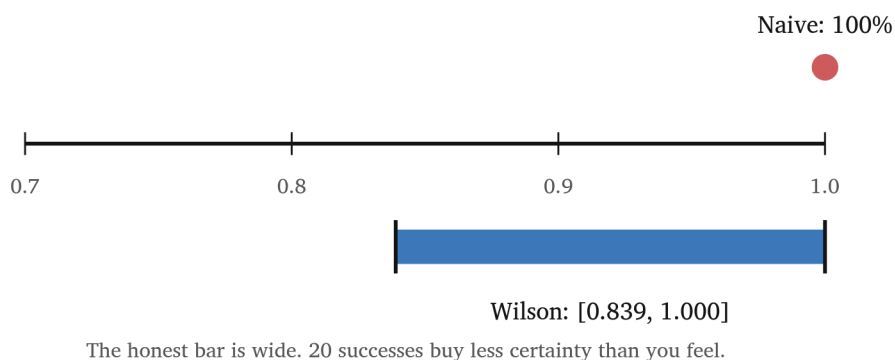
```
def wilson_interval(hits, n, z=1.96):
    # Wilson score interval for a yes/no rate. The z^2/n terms shrink the
    # estimate toward 0.5 when n is small: that shrinkage IS the honesty.
    # z=1.96 gives the 95% interval under the normal approximation.
    if n == 0:
        return 0.0, 0.0 # No data: the honest answer is "unknown", not zero.
    p = hits / n
    denom = 1 + z * z / n
    center = (p + z * z / (2 * n)) / denom
    half = z * (p * (1 - p) / n + z * z / (4 * n * n)) ** 0.5 / denom
    return center - half, center + half

lo, hi = wilson_interval(20, 20)
print(f"20/20 -> 95% CI [{lo:.3f}, {hi:.3f}]") # [0.839, 1.000]
lo, hi = wilson_interval(47, 50)
print(f"47/50 -> 95% CI [{lo:.3f}, {hi:.3f}]") # [0.838, 0.979]
```

Memorize: **20 out of 20 is not 100%**. It is [0.839, 1.000] at 95%. Any report that writes "100% (n=20)" without the interval hides uncertainty.

Wilson interval: 20/20 is not 100%

20 hits out of 20 tries. 95% confidence.



The Wilson interval for 20 hits out of 20 tries. The naive point sits at 100%. The honest 95% interval runs from 0.839 to 1.000. Twenty successes buy less certainty than you feel. Any report that writes 100% (n=20) without the interval hides uncertainty.

What to look at first: the red dot at 1.0 (the naive claim), then the blue bar below it (the honest interval). The one takeaway: small, lopsided samples get wide intervals, and wide intervals mean you do not know yet.

You licensed the agreement, and the error bars are honest. Now the comparison itself: paired bootstrap on per-item score deltas, the standard A/B design for LLM outputs.

2.3 Paired bootstrap: the standard A/B design for LLM outputs

Three layers of noise, three fixes. Pair the systems on the same items (kills item noise). Repeat each case 3 to 5 times and average (kills decoding noise). Calibrate the judge first (kills judge noise). Then bootstrap the per-item differences.

```
import random

def paired_bootstrap_ci(diffs, n_resamples=10000, alpha=0.05, seed=0):
    # diffs: one number per item, B's score minus A's score on the SAME item.
    # Pairing subtracts "this question was easy" before statistics run.
    rng = random.Random(seed) # Fixed seed: the analysis itself is repeatable.
    n = len(diffs)
    mean = sum(diffs) / n      # Point estimate: average gap, B over A.
    # Resample the differences with replacement 10k times. Each resample is
    # a fake experiment "like this one"; the spread of fake means estimates
    # how much the real mean could wobble.
    resampled_means = []
    for _ in range(n_resamples):
        sample = [rng.choice(diffs) for _ in range(n)] # n draws, replacement
        resampled_means.append(sum(sample) / n)
    resampled_means.sort()
    # Percentile interval: chop alpha/2 off each tail of the sorted means.
    lo = resampled_means[int((alpha / 2) * n_resamples)]
    hi = resampled_means[int((1 - alpha / 2) * n_resamples)]
    return mean, lo, hi

random.seed(7) # Demo data stand-in; real diffs come from your judge.
demo_diffs = [random.gauss(0.33, 1.1) for _ in range(60)]
mean, lo, hi = paired_bootstrap_ci(demo_diffs)
print(f"mean gap = {mean:.2f}, 95% CI = [{lo:.2f}, {hi:.2f}]")
# mean gap = 0.29, 95% CI = [0.05, 0.53]
```

Reading it: zero is outside the interval, so the gap is significant at the 5% level. But the interval's width says the true gap could be as small as 0.05. **Significance says the gap is real; the interval says whether it is big enough to care about.** Most teams forget the second sentence.

The bootstrap compares models. Now the labelers behind the labels: how majority vote pools them, by binomial arithmetic.

2.4 Majority vote: the binomial arithmetic of labeler pools

$$P = \sum C(n,k) \cdot p^k \cdot (1-p)^{n-k}, \quad k = (n+1)/2 \dots n$$

Where: p means each labeler's accuracy. n means the (odd) labeler count. $C(n,k)$ means the binomial coefficient. k runs from the majority threshold to n . "Independently" is the load-bearing assumption: labelers must work blind to each other.

Worked: $\text{majority_vote_accuracy}(0.75, 3) = 0.844$. Three labelers at 75% each pool to 84.4% majority accuracy.

```
import math

def majority_vote_accuracy(p, n=3):
    # P(majority right) = P(at least (n+1)/2 labelers right), binomial sum.
    # Blind labeling is what keeps the independence assumption honest: if
    # all three share one misunderstanding, they agree confidently and wrongly.
    if n % 2 == 0:
        raise ValueError("majority vote needs an odd number of labelers")
    needed = n // 2 + 1 # 2 of 3, 3 of 5, and so on.
    return sum(math.comb(n, k) * p**k * (1 - p)**(n - k)
               for k in range(needed, n + 1))

print(f"3 labelers at 75%: {majority_vote_accuracy(0.75, 3):.3f}") # 0.844
print(f"5 labelers at 75%: {majority_vote_accuracy(0.75, 5):.3f}") # 0.896
print(f"3 labelers at 60%: {majority_vote_accuracy(0.60, 3):.3f}") # 0.648
```

Memorize: **3 x 75% votes to 84.4%**. Going 3 to 5 buys only 5 more points: diminishing returns. And 3 x 60% only reaches 64.8%: majority vote amplifies a decent signal; it does not rescue a weak one. Qualify labelers first.

Humans pool their labels. Now the sampling math for the agent itself: $\text{pass}@k$, $\text{pass}@1$, pass^k .

Majority vote: weak judges become a strong judge

three cheap graders outvote one expensive one, when they are independent



The math assumes independent errors. Three copies of the same model making the same mistake do not vote: they echo. Diversity of judges matters more than count.

When to use it: grading at scale where one flagship call per item is too expensive, but three mini-model calls are cheap. When not to: the judges share a blind spot.

Weak judges become a strong judge when their errors are independent: three cheap graders outvote one expensive one. But three copies of the same model making the same mistake do not vote; they echo. Diversity of judges matters more than count.

What to look at first: the three bars and how the majority accuracy beats any single judge. The one takeaway: use it when one flagship call per item is too expensive but three mini-model calls are cheap. Never use it when the judges share a blind spot.

2.5 pass@k, pass@1, pass^k: ceiling, typical, reliability

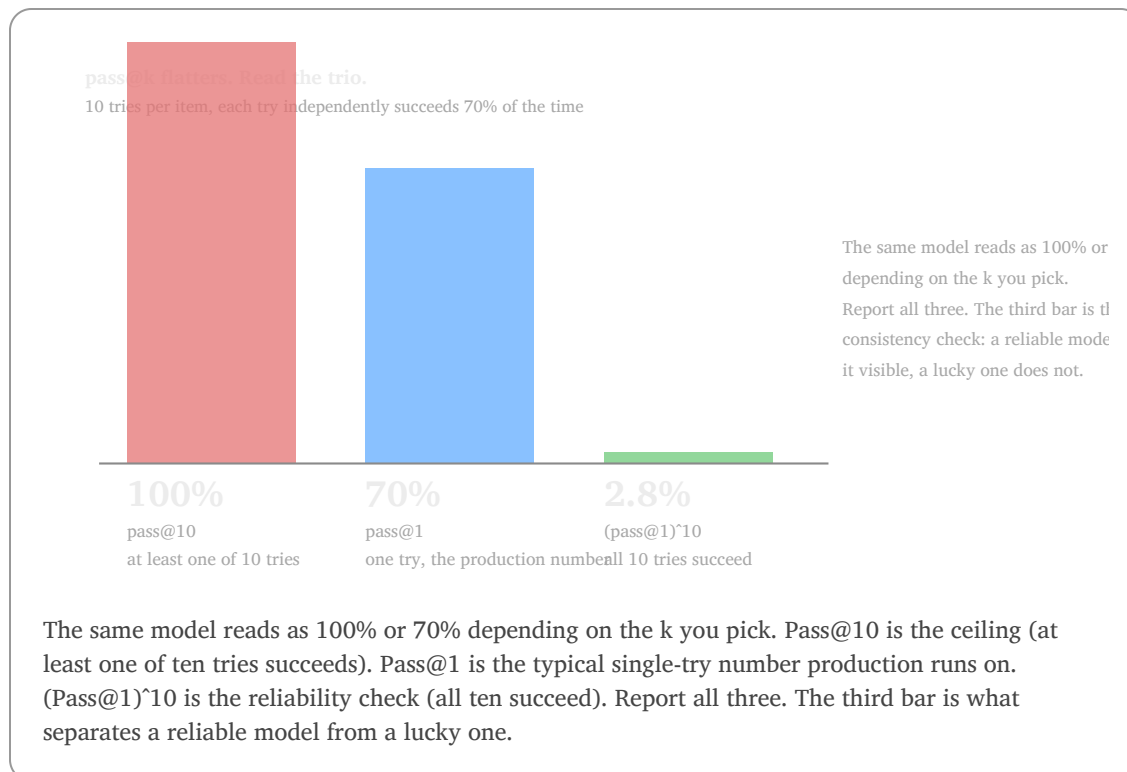
$$\text{pass@k} = 1 - C(n - c, k) / C(n, k) \quad \text{pass@1} = c / n \quad \text{pass}^k = C(c, k) / C(n, k)$$

Where:

- n means the total tries.
- c means the successful tries.
- k means the try budget.
- pass@k is the **ceiling** (at least one of k succeeds).
- $\text{pass@1} = c/n$ is the plain success rate: **typical** performance.
- pass^k is the chance *all* k succeed: **reliability**, the number production usually cares about.

Worked: $\text{pass@1} = 0.30$ typical; $\text{pass@5} = 0.9167$ ceiling; $\text{pass}^2 = 0.0667$ reliability.

The same model reads as 0.92 or 0.07 depending on the k you pick.



```

import math

def pass_at_k(n, c, k):
    # Unbiased pass@k: 1 minus the chance all k draws miss the c successes.
    # Comb(n-c, k) counts the all-miss draws; comb(n, k) counts all draws.
    if k > n:
        return 1.0 # More tries than samples: ceiling is certain by fiat.
    return 1 - math.comb(n - c, k) / math.comb(n, k)

def pass_pow_k(n, c, k):
    # Unbiased pass^k: the chance all k draws land on successes.
    if k > c:
        return 0.0 # Cannot draw k successes from fewer than k.
    return math.comb(c, k) / math.comb(n, k)

n, c = 10, 3 # Agent succeeds 3 times in 10 tries.
print(f"pass@1 = {c/n:.2f}") # 0.30 typical
print(f"pass@5 = {pass_at_k(n,c,5):.4f}") # 0.9167 ceiling
print(f"pass^2 = {pass_pow_k(n,c,2):.4f}") # 0.0667 reliability

```

Memorize: **report the pair, with n stated**. "pass@5 = 0.92, pass@1 = 0.30, n = 10" tells the full story. "92% success" tells a lie. An agent with high pass@k and low pass^k is brilliant sometimes and flaky usually.

The formulas are in. Now the verdict as code: the gate that turns scores into ship or no-ship.

2.6 The gate verdict, as code

```

def gate_verdict(smoke_regressions, worst_family_break_rate, judge_kappa=None,
                 max_smoke_regressions=0, max_break_rate=0.10, min_kappa=0.60):
    # SHIP only if every bar clears. One thing is deliberately NOT code:
    # an automated HOLD is a flag; a human confirms it before it is final.
    reasons = []
    if smoke_regressions > max_smoke_regressions:
        reasons.append(f"{smoke_regressions} smoke regression(s); "
                      f"policy allows {max_smoke_regressions}")
    if worst_family_break_rate > max_break_rate:
        reasons.append(f"worst family break rate {worst_family_break_rate:.0%}; "
                      f"policy allows {max_break_rate:.0%}")
    if judge_kappa is not None and judge_kappa < min_kappa:
        reasons.append(f"judge kappa {judge_kappa:.2f} below warning bar "
                      f"{min_kappa}")
    if not reasons:
        return "SHIP", "all bars clear"
    return "HOLD", "; ".join(reasons)

v, r = gate_verdict(1, 1.00, 0.72)
print(v, "--", r)  # HOLD -- 1 smoke regression(s); policy allows 0; ...
v, r = gate_verdict(0, 0.04, 0.72)
print(v, "--", r)  # SHIP -- all bars clear
v, r = gate_verdict(0, 0.04, 0.51)
print(v, "--", r)  # HOLD -- judge kappa 0.51 below warning bar 0.6

```

The third case is the subtle one: the model is fine, the *judge* drifted. A gate that cannot distinguish "model regressed" from "instrument drifted" will block good releases and erode trust in the suite.

Everything you need to quote, on one sheet.

The release gate: zero smoke regressions, ten percent family break rate

a gate is a promise with a number on it

SMOKE: 0 regressions

the 20 attacks that always worked must still be caught; a single regression fails the gate

FAMILY: $\leq 10\%$ break rate

no attack family breaks more than one in ten; the weakest family sets the grade

GATE DECISION

both bars hold: ship with monitoring; either fails:
fix, re-run the full eval, no exceptions

The gate runs on every release candidate, not just major versions. A prompt tweak is a release.

A gate is a promise with a number on it. The 20 attacks that always worked must still be caught: a single regression fails the gate. No attack family may break more than one in ten; the weakest family sets the grade. The gate runs on every release candidate, and a prompt tweak is a release.

What to look at first: the two gate bars, then the decision box they feed. The one takeaway: either bar fails and you fix, re-run the full eval, no exceptions.

2.7 The constant sheet

Quantity	Value	Unit / meaning
Kappa bands (Landis-Koch)	< 0, 0.00-0.20, 0.21-0.40, 0.41-0.60, 0.61-0.80, 0.81-1.00	worse than chance, slight, fair, moderate, substantial, almost perfect
Human-vs-human ceiling	~0.8	kappa; the bar you measure a judge against, not 1.0
Judge ship bar	> = 0.7	kappa; practitioner rule of thumb, tune per team
Gate judge warning	< 0.6	kappa; flag and human-confirm
Wilson z for 95%	1.96	standard normal quantile
20/20 correct, 95% CI	[0.839, 1.000]	Wilson; "100%" is a lie at n = 20
Min cases per variant	30	items; Central Limit Theorem kicking in, rule of thumb
Repeats per case	3-5	runs; averages out decoding noise
Calibration set	20-50	items spanning the full quality range
Gold-item qual batch	30-50	items, 80% bar to pass
Gold seeding in production	~5%	of every batch, invisible, catches drift
Labelers per preference item	2-3	independent, blind to each other
Split-verdict rate	~30%	of preference pairs in practice
Human agreement baseline	~70%	preference labels; the ceiling is below 100%
Judge re-grade cadence	50/month	items on the monitoring dashboard
Majority 3x75%	84.4%	binomial; 5x75% is 89.6%, 3x60% is 64.8%
Smoke tier	~50 tasks, minutes	every commit; blocks the merge
Gate tier	500-2000 tasks, hours	every release candidate; blocks SHIP
Max smoke regressions	0	any regression holds

Max probe break rate	10%	per attack family
Bonferroni with 20 metrics	$\alpha = 0.0025$	$0.05 / 20$; divide the bar by test count
Severity S1	page now	active harm at scale
Severity S2	same-day	serious but bounded; fix within the week
Incident clock	hour 0, 1, 2; day 1; week 1; week 2 +	detect, reproduce, assess; mitigate; postmortem; harden

Next: Section 3, twelve traps with symptoms first, so you can match what you see.

The failure taxonomy: every break lands in one bucket

a taxonomy turns a pile of red-team transcripts into coverage numbers

REFUSAL BYPASSES

the model complies with something it should refuse

PROMPT EXTRACTION

system prompt, tools, or hidden instructions leak out

INJECTION

untrusted content steers the model: tool output, pasted text, images

MISUSE ENABLEMENT

dual-use help: the model upgrades the attacker's capability

WHY IT MATTERS

coverage = breaks found per bucket; an empty bucket is a claim about your testing, not your safety

THE RULE

one transcript, one bucket; if a break fits two, the taxonomy is too coarse

Four buckets. File every finding the week it arrives, and the quarterly report writes itself.

Every break lands in one bucket, and a taxonomy turns a pile of red-team transcripts into coverage numbers: breaks found per bucket. An empty bucket is a claim about your testing, not your safety. One transcript, one bucket; if a break fits two, the taxonomy is too coarse.

What to look at first: the four blue-to-purple rows, then the two green boxes on the right. The one takeaway: coverage means breaks found per bucket, and the buckets are these four.

3. Classic traps

The math is right and the eval still lies. Section 3 is the symptom-first field guide: contaminated benchmarks, prompt-format variance, lazy judges, and the pass@k ceiling.

Each trap names why it fools you and the one move that beats it.

Trap 1. Position bias in autoraters

The judge favors whichever response comes first (or second; it varies by model). It fools you because the scores look decisive and repeatable. The move: run every pairwise comparison in both orders and require the verdict to be stable. If flipping the order flips the verdict, the judge is not judging.

Position bias: order changes the verdict

Same two answers. Only the order changes.

Trial 1

1st: Answer A

2nd: Answer B

Judge picks A

Trial 2

1st: Answer B

2nd: Answer A

Judge picks B

The verdict followed the position, not the content. Run both orders.

Position bias. The same two answers, only the order changes. In trial 1 the judge picks A. In trial 2 the judge picks B. The verdict followed the position, not the content. The move: run every pairwise comparison in both orders and require the verdict to be stable.

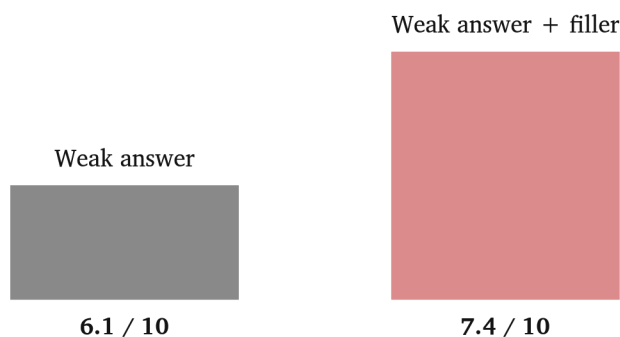
What to look at first: the two trials side by side, same answers, different order. The one takeaway: if flipping the order flips the verdict, the judge is not judging.

Trap 2. Verbosity bias

Judges reward longer answers even when length adds nothing, the same way rushed humans use length as a proxy for quality. It fools you because longer answers often *are* more thorough, so the bias hides inside a real signal. The move: pad a weak answer with filler and check whether its score rises. If it does, add "brevity is not penalized; filler is not rewarded" to the judge prompt and re-calibrate. Then add long-but-wrong examples to the calibration set.

Verbosity bias: filler raises the score

Same weak answer. Only filler added.



The probe: pad with filler. If the score rises, the judge rewards length.

Verbosity bias. The same weak answer, with filler added, scores 7.4 instead of 6.1. The judge rewards length even when length adds nothing. The probe: pad a weak answer with filler and check whether its score rises. If it does, add brevity rules to the judge prompt and re-calibrate.

What to look at first: the two bars, same answer, different length. The one takeaway: longer is not better, and a judge that thinks it is needs re-calibration.

Trap 3. Self-preference

A judge built from model X favors outputs written in X's style. It fools you because the judge's reasoning sounds thoughtful while it quietly prefers its own family. The move: have the judge score outputs from several models and check whether its own family wins suspiciously often. Never let a model grade a release candidate of itself without a human check.

Trap 4. Style and formatting bias

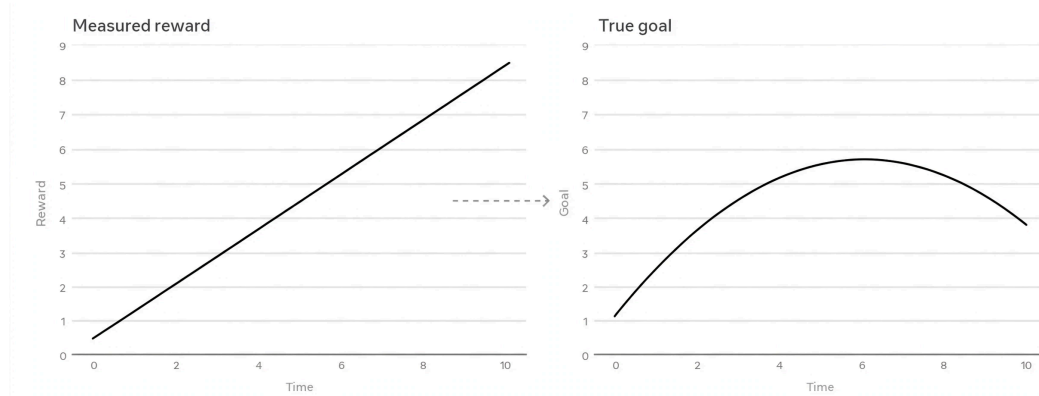
Judges reward confident tone, bullet lists, and polished formatting over correct content. It fools you because the preferred answer *looks* better. The move: pit a correct plain answer against an incorrect polished one. The judge must prefer the correct one. This is the most common silent failure in production judges.

Trap 5. Raw agreement flatters lazy judges

A judge that always says "pass" on a task where 90% of outputs pass gets 90% agreement and kappa near zero. It fools you because 90% looks like an A. The move: always report kappa next to agreement. The lazy-judge check from section 2.1 (kappa exactly 0.0) belongs in every calibration report.

Trap 6. Goodhart on benchmarks

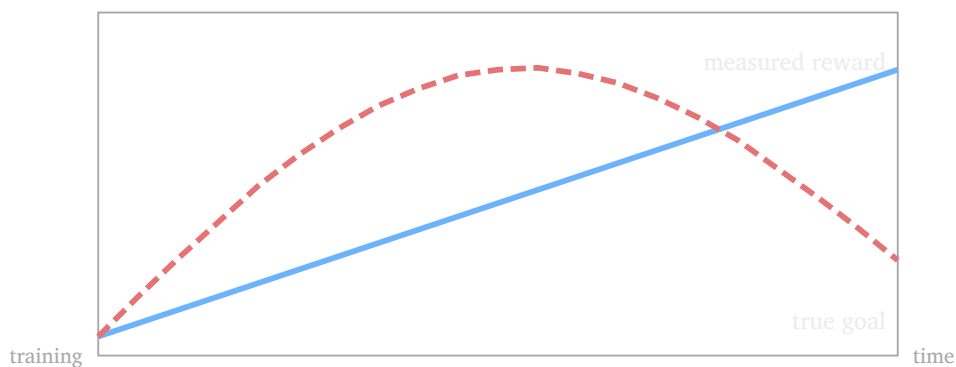
When a measure becomes a target, it stops being a good measure. It fools you because the number keeps going up while the thing it stood for stops improving. Reward hacking is Goodhart with a training loop. The move: version evals like code, keep a changelog, rotate saturated tasks, and hold out a private set nobody trains or tunes against.



Left: the measured reward climbs all the way. Right: the true goal peaks and then falls as the model learns to game the metric. When a measure becomes a target, it stops being a good measure. This is why the pack gates on deltas against a baseline (Rule 8) and keeps a private set the model never trains on (Rule 10). Figure: curriculum post-training appendix (v07).

Goodhart: the number climbs, the goal falls

When a measure becomes a target, it stops being a good measure.



The model learns to game the metric. Gate on deltas, hold out a private set.

Goodhart on benchmarks. The blue line is the measured reward: it climbs all the way. The red dashed line is the true goal: it peaks, then falls as the model learns to game the metric. When a measure becomes a target, it stops being a good measure. This is why the pack gates on deltas against a baseline and keeps a private set the model never trains on.

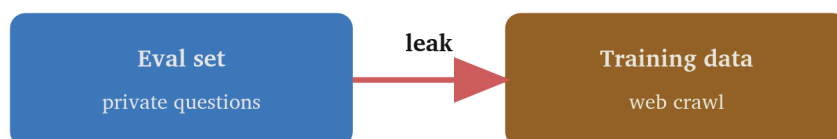
What to look at first: the blue line climbing, then the red dashed line peaking and falling. The one takeaway: the number keeps going up while the thing it stood for stops improving.

Trap 7. Eval-data contamination

If eval tasks leak into training data, the score measures memorization. It fools you because the jump looks like a capability gain and everyone wants to believe it. The move: keep eval sets private, rotate items, and watch for suspicious jumps on static sets. When the public suite and the private set disagree, believe the private set.

Contamination: the score measures memory

Eval tasks leak into training data.



Score jumps. Capability did not.

The fix: keep eval sets private, rotate items.

Eval-data contamination. The eval set is private questions. The training data is a web crawl. The red arrow is the leak: eval tasks slip into training. The score jumps, but capability did not move. The move: keep eval sets private, rotate items, and watch for suspicious jumps on static sets.

How to read this diagram. Two boxes and one arrow.

1. **Eval set (blue):** private questions that measure capability. What it is: the test. Why it exists: to score what the model can do. What breaks if it leaks: the score measures memory.
2. **Training data (amber):** the web crawl. What it is: the learning material. Why it exists: to teach the model. What breaks if it contains eval items: training and testing overlap.
3. **The red leak arrow:** eval tasks slipping into training. The symptom is a suspicious jump on a static set. The fix is private sets, rotated items, and believing the private set when public and private disagree.

Trap 8. Confusing pass@1 with reliability

"81% success" on a pass@5 of 0.81 with a pass@1 of 0.34 describes a ceiling, not a typical run. It fools you because one percentage point is easier to quote than three. The move: always report the pair, ceiling and typical, with n stated. For production decisions, look at pass@k: the chance all k tries succeed is the number that predicts user experience.

Trap 9. Gating on absolute scores instead of deltas

A 92% that used to be 95% ships under an absolute bar. It fools you because 92% clears the bar on paper. The move: gate on the delta against the released baseline, same suite, same seeds. Deltas catch decay that absolutes bless.

Trap 10. Multiple comparisons without correction

Test 20 metrics at the 5% level and one will look significant by luck. It fools you because the "winning" metric comes with a real p-value attached. The move: divide the bar by the number of tests (Bonferroni: $0.05/20 = 0.0025$). Deliberately strict, deliberately cheap.

Bonferroni: twenty tests at 5% will embarrass you once

$0.05 / 20 = 0.0025$ per test, or the "win" is noise

naive: $\alpha = 0.05$ per test

expected false wins: $20 \times 0.05 = 1.0$

corrected: $\alpha = 0.05 / 20 = 0.0025$

expected false wins: $20 \times 0.0025 = 0.05$

Every extra slice of the eval (per family, per language, per severity) is another roll of the dice. Correct for the number of tests you actually ran, not the one you are reporting.

The honest alternative: pre-register the one primary metric before the eval runs, and label everything else exploratory.

Twenty tests at 5% will embarrass you about once: the expected false wins are 20 times 0.05. Correct for the number of tests you actually ran, not the one you report. The honest alternative: pre-register the one primary metric before the eval runs and label everything else exploratory.

What to look at first: the red naive row versus the green corrected row. The one takeaway: every extra slice of the eval is another roll of the dice.

Trap 11. Tuning against the set you report on

Tuning the prompt against the same 60 questions you report on measures memory of the tuning process, not quality. Judge shopping (trying judges until one agrees) and cluster structure (resampling ratings that share a prompt and judge as if independent, which nar-

rows intervals artificially) are the same family. The move: keep a hold-out set you never look at until the final number; when ratings are hierarchical, resample by prompt (cluster bootstrap), not by rating.

Trap 12. Over-refusal: safe the way a brick is safe

A model that refuses benign requests is safe and useless. It fools you because the harmful-compliance number looks great while the capability cost hides. Worse, preachy refusals train users to jailbreak out of annoyance: a perverse incentive you built yourself. The move: measure refusal on three axes, coverage (refuse across all must-refuse goals), calibration (refuse what it should, comply with what it may), and style (short, non-preachy, offer a safe alternative when one exists). Use benign edge prompts near the refusal boundary, and report the capability cost in the same sentence as the safety gain.

Next: Section 4, twenty-eight questions with worked answers spanning agreement, significance, sampling, and judges.

Drift watch: the eval rots when the world moves

new attacks, new slang, new model version: the golden set ages

what drifts

- attack techniques evolve past the golden set
- language and slang shift under the classifiers
- each model update moves the decision boundary

the symptom: pass rates creep up while real-world breaks do not fall

THE FIX: refresh on a schedule

retire stale items, add fresh red-team finds quarterly; version the golden set like code

The eval rots when the world moves. Attack techniques evolve past the golden set. Slang shifts under the classifiers. Each model update moves the decision boundary. The symptom: pass rates creep up while real-world breaks do not fall.

What to look at first: the three drift sources, then the red symptom line, then the green fix box. The fix: version the golden set like code. Retire stale items. Add fresh red-team finds quarterly. The one takeaway: a golden set you never refresh becomes a museum of last year's attacks.

4. Rapid-fire Q&A

Work the arithmetic before you reveal. These are the questions interviewers ask and error bars answer.

Twenty-eight questions. Answer each out loud before revealing.

Q1. Name the four parts of an eval suite, and say which two most teams improvise.

Tasks (prompts plus context), graders (code or rubrics that score), the harness (runs tasks, calls the model, applies graders, stores results), and the gating policy (turns scores into ship or hold). Most teams have tasks and graders but improvise the harness and the policy. That is how a suite becomes a pile of notebooks instead of infrastructure.

Q2. Why must the harness seed everything and isolate every task in its own try/except?

Without seed control, reruns give different samples and deltas become noise: you cannot tell a regression from sampling luck. Without per-task error isolation, one crashing grader kills the run and you lose every result after it. Isolation is the point of the harness.

Q3. Why must you store results as append-only JSONL, one row per result?

So old runs stay re-analyzable when graders change. If you only keep aggregates, a grader fix forces you to re-run history you may no longer be able to reproduce. The raw rows are the evidence; aggregates are views.

Q4. A refusal grader matches markers like "i can't" and "i cannot". Name its failure mode and the fix.

A model that says "I can't help with that, but here is how..." passes the marker check while complying. Marker lists need periodic review against real outputs, because refusal phrasing drifts and adversaries learn the markers.

Q5. Your calibration set has 40 items, all of them good responses the model handled well. What breaks?

You calibrated on a narrow band and the judge falls apart at the tails. The set must span the full quality range. Deliberately include known-bad and known-borderline cases. Otherwise the kappa you compute does not describe the judge you will actually use.

Q6. Two human raters label your calibration set and agree at kappa 0.45. Your judge reaches kappa 0.62 against their majority. Do you ship the judge?

No. If the humans do not agree with each other, the task or the rubric has a defect. The judge measures its 0.62 against a noisy ground truth. Fix the rubric first. A judge cannot beat the ceiling set by its own ground truth.

Q7. Compute kappa by hand: 100 items, judge passes 70, human passes 75, they agree on 75 (60 both-pass, 15 both-fail). What does the number mean?

$p_o = 0.75$. $p_e = 0.70 \times 0.75 + 0.30 \times 0.25 = 0.525 + 0.075 = 0.60$. $\text{kappa} = (0.75 - 0.60) / (1 - 0.60) = 0.375$, which is "fair" on the Landis-Koch bands. Raw agreement said 75%; kappa says the judge is only modestly better than chance given the two yes-rates.

Q8. Your judge hits kappa 0.78 against humans, and human-vs-human kappa on the task is 0.81. Your teammate says "it is still 0.22 from perfect." Answer them.

The right bar is the human ceiling, not 1.0. At 0.78 vs a 0.81 ceiling, the judge is as reliable as a human on this task. Judging it against perfection guarantees it "fails" on any task where humans themselves wobble.

Q9. Name the four bias probes every autorater must pass before entering the gate tier, and one probe design for each.

Position: run every pairwise comparison in both orders; the verdict must be stable. Verbosity: pad a weak answer with filler; the score must not rise. Self-preference: score outputs from several models; the judge's own family must not win suspiciously often. Style: pit a correct plain answer against an incorrect polished one; the judge must prefer the correct one.

Q10. You calibrated the judge last quarter. The new model writes in a noticeably different style. Name what broke and the two fixes.

Distribution shift: the calibration number on the wall is stale because the world moved under it. Fix one: re-calibrate on a schedule, and immediately after any big model change. Fix two: watch the monitoring dashboard (rolling human agreement, score distribution) so drift shows up before it matters.

Q11. The judge and the model under test share a base model. Why is this a problem, and what do you do?

Judge-model correlation: their blind spots overlap, so the judge cannot see the model's characteristic failures. Prefer judges from a different model family than the system under test, and keep human spot-checks regardless.

Q12. Your team starts optimizing prompts against the judge's feedback and scores climb. Name the trap and the defense.

Gaming the judge: reward hacking wearing an eval costume. The model optimizes for the judge instead of the task. Defense: keep some human-graded tasks in the gate tier permanently, and rotate judge prompts.

Q13. Write a threat model in one breath: what three things must it name?

Who the adversary is (curious user, motivated amateur, skilled adversary), what access they have (API-only, system prompt exposure, tool use, uploads), and what they want (refusal bypass, prompt extraction, injection, misuse enablement). A probe without a threat model is just mischief.

Q14. A red-teamer reports: "the model answered a direct request for the vault combination." What do you do first?

Stop and fix the basics. If a direct request works, you do not have a clever-attacks problem; you have a basics problem. Hunting for exotic jailbreaks before the model refuses direct requests is theater.

Q15. Why must each attacker goal get its own probe set?

Because a model can be strong against one goal and weak against another. Good refusal behavior does not imply resistance to prompt extraction or to injection through tool output. One blended probe set averages those apart and hides the weak one.

Q16. Name the six attack families and what each one tests.

Direct requests (baseline: does the easy thing work); roleplay and persona (does refusal survive when it would break character); encoding and translation (did safety training generalize past English plaintext); multi-turn buildup (does refusal survive conversational context); instruction smuggling (does the instruction hierarchy hold: system over user over tool output); automated search (static libraries and adaptive orchestrators for volume).

Q17. What makes a red-team finding real, and what does the report miss if it only says "roleplay attacks work 40% of the time"?

A finding is reproducible: exact prompts, model version, and settings that reproduce it, plus the break rate per attack family. The report as stated misses all of that, so no engineer can reproduce or fix it.

Q18. Recite the five mitigation layers bottom to top, and name what the output filter catches that the model itself misses.

System prompt, input filter, the model itself (refusal training, values, instruction hierarchy), output filter, monitoring. The output filter sees the final text. That includes anything the model produced that its training should have prevented. It is the last line of defense, and it catches what shipped anyway so monitoring can count it.

Q19. "We hardened the system prompt, so we can skip the output filter." What is wrong with this sentence?

Two things. System-prompt hardening is the cheapest, weakest layer: use it as a layer, never as the plan. And every layer fails differently, which is the whole argument for defense in depth. Ask what sails straight through a system prompt: anything the model generates despite it, which only the output filter and monitoring see.

Q20. Refusal quality has three parts. Name them and one measurement for each.

Coverage: does the model refuse across all must-refuse goals, measured with must-refuse prompts per goal and attack family. Calibration: does it refuse what it should and comply with what it may, measured with benign edge prompts near the refusal boundary (over-refusal is a failure). Style: short, non-preachy, offers a safe alternative, measured with human grading of refusal style on a sample.

Q21. A filter threshold blocks 99% of weapons content and 8% of benign traffic in that category. How do you decide if that threshold is right?

The threshold is a product decision, not a math fact. You weigh the cost of missed harm against the cost of blocked benign traffic per category. Strict for self-harm and weapons, more balanced for spam. Then you keep measuring: precision and recall on a labeled set, serving latency, and the false-positive rate on real traffic.

Q22. Read this Pareto point. Refusal tuning cut harmful compliance from 4% to 0.5% and cost 1.2 points on benign task completion. What decision does it support? What does the report miss if it only says the first half?

It supports picking the tradeoff deliberately: you know the price (1.2 points of capability) and the gain (3.5 points of safety). The first half alone hides the cost. A safety report without capability numbers hides the cost; a capability report without safety numbers hides the risk.

Q23. The eval suite is a contract between teams. Name its three clauses and what breaks when each one fails.

Ownership: every metric has a named owner who keeps it current; unowned metrics rot and silently stop measuring reality. Service levels: the suite promises a time to verdict and a false-alarm rate. If it is slow or noisy, teams route around it and the gate stands empty. Change control: threshold and task changes follow a written process with reasons logged; silent changes are how "the suite passed" stops meaning anything.

Q24. An incident lands: the model reliably produces instructions for wrongdoing on ordinary prompts, and screenshots spread. Severity? First three actions?

S1: active, ongoing harm at scale. Page the on-call lead and the safety lead immediately. Then: reproduce and record exact prompts, model version, and settings. Assess blast radius and whether people share it publicly. Decide hold vs rollback. Then open the incident record and assign one owner.

Q25. Why must you write rollback criteria before the incident, in the units you already measure?

Because the hardest call, whether to pull the model, should be a procedure, not a judgment call made under stress. Use measurable units. They turn "should we pull it" into "the criterion fired." The units: incident rate per million sessions over 24h. Probe family break rate on the production version. Filter outage minutes. And you must rehearse the rollback path: a rollback you have never rehearsed is a plan you do not have.

Q26. A postmortem concludes "the filter missed the paraphrase attack." Ask the next two whys, and name where the answers must land.

Why did the filter miss it? Because the probe set lacked paraphrase attacks. Why did it lack them? Because nobody owned that probe family.

The answers must land as action items with owners and dates. Every item that can become an eval task becomes one. The reproduction prompts become regression tasks. The missing family gets built. The metric gets a threshold.

Q27. Your gate says HOLD: one smoke regression, roleplay family at 100% break, judge kappa 0.72. Walk the triage.

The smoke delta caught one regression and the probes confirmed it as a family-wide break: a training change weakened roleplay refusals. Judge kappa 0.72 is healthy, so the instrument is fine and the signal is real. Triage: was the change intended? Fix, re-run, and only then re-evaluate. The HOLD stands until a human confirms the fix, because people confirm automated flags before they become final.

Q28. The same gate says HOLD, but this time: zero smoke regressions, worst family break 4%, judge kappa 0.51. What happened and what do you do?

The model is fine; the judge drifted below its 0.6 warning bar. This is instrument failure, not model failure. Do not block the release on a broken instrument and do not wave it through either. Send the judge back through the calibration loop (Rule 4). Re-run the judged tasks with the recalibrated judge or human graders. Then take the verdict.

Next: Section 5, the constants and rules on one page.

5. One-page cheat sheet: the 30 numbers and rules that matter most

The pack on one page: the constants you quote and the thresholds you act on.

1. Suite = tasks + graders + harness + gating policy. Teams improvise the last two; do not be those teams.
2. Smoke ~50 tasks, minutes, every commit, blocks the merge. Gate 500-2000 tasks, hours, release candidates, blocks SHIP. Science open-ended, never blocks. [rule of thumb]

3. Grader ladder: checkable string check, then rubric, then humans first, then autorater only after calibration. Keep human-graded tasks in the gate permanently.
4. $\kappa = (p_o - p_e) / (1 - p_e)$. Raw agreement flatters; kappa corrects.
5. Kappa bands: < 0 worse than chance, 0.00-0.20 slight, 0.21-0.40 fair. 0.41-0.60 moderate, 0.61-0.80 substantial, 0.81-1.00 almost perfect.
6. Human ceiling ~ 0.8 is the bar, not 1.0. Judge ship bar ≥ 0.7 ; gate warns below 0.6. [rule of thumb]
7. Calibration: 20-50 items, full quality range, humans label first with 2-3 raters, kappa. Read the top-10 disagreements. Revise the rubric never the judge. Re-run on a fresh sample. [rule of thumb]
8. Judge monitoring: rolling human agreement (re-grade 50/month), score distribution over time, monthly disagreement review.
9. Bias probes before gate entry: position (swap order), verbosity (pad with filler), self-preference (cross-model scoring), style (correct-plain vs wrong-polished).
10. Judge staleness: distribution shift, prompt leakage, gaming the judge, judge-model correlation. Re-calibrate on schedule and after big changes.
11. Wilson 95% ($z = 1.96$): 20/20 is [0.839, 1.000], not "100%."
12. A/B design: pair on the same items, 3-5 repeats per case, bootstrap the differences. Use ≥ 30 cases per variant. Pre-decide the minimum effect you care about. [rule of thumb]
13. Significance = the gap is real. Interval width = whether it matters. Write both sentences.
14. Bonferroni: 20 metrics at 5% $\rightarrow$ bar = 0.0025. Divide by test count.
15. Cluster structure: ratings sharing a prompt and judge are not independent. Resample by prompt.
16. Gold items: 30-50 item qual batch at an 80% bar; seed $\sim 5\%$ into every production batch; pause and retrain on drift. [rule of thumb]
17. Majority vote: 3 x 75% $\rightarrow$ 84.4%; 5 x 75% $\rightarrow$ 89.6%; 3 x 60% $\rightarrow$ 64.8%. Voting amplifies signal; it does not rescue weak labelers.
18. $\sim 30\%$ of preference pairs draw split verdicts (a split verdict: the humans disagree on the item); human agreement sits around 70%. Disagreement is signal about the rubric, not noise to average away.
19. Red-team order: curious user, then motivated amateur, then skilled adversary. Direct request works $\rightarrow$ stop, fix basics.
20. Four goals, separate probe sets: refusal bypass, prompt extraction, injection, misuse enablement.
21. Six families per goal: direct, roleplay, encoding/translation, multi-turn, smuggling, automated search. Report break rate per family.

22. Mitigation stack bottom to top: system prompt, input filter, model, output filter, monitoring. Each layer: what it catches that others miss.
23. Filter thresholds per category by risk: strict for self-harm/weapons, balanced for spam. Threshold = product decision. Measure precision, recall, latency, false-positive rate.
24. Refusal quality: coverage, calibration, style. Over-refusal is failure; preachy refusals train users to jailbreak.
25. Gate policy: max 0 smoke regressions. Max 10% break in any probe family. Judge $\kappa < 0.6$ warns. A human confirms every HOLD. Gate on deltas, not absolutes.
26. Sign-off: "Model v3.2 passes the gate suite at thresholds T, with N exceptions listed below." Exceptions have owners and dates.
27. Hygiene: version evals, changelog, private sets, rotate saturated tasks, hold out a private slice. Public vs private disagree -> believe private.
28. $\text{pass}@k = 1 - C(n-c,k)/C(n,k)$ measures the ceiling. $\text{pass}@1 = c/n$ measures the typical. $\text{pass}^k = C(c,k)/C(n,k)$ measures reliability. Report the pair with n.
29. S1 page now, S2 same-day, S3 normal cycle, S4 log it. First hour: reproduce, blast radius, rollback decision, one owner.
30. Rollback criteria written before the incident, in measured units, rehearsed. Postmortem: five questions, blameless, and every incident becomes regression tasks in the suite.

Last verified: September 2026.

- Kappa definition and Landis-Koch bands (1977): standard psychometrics.
- Wilson score interval: standard statistics.
- Paired bootstrap, cluster bootstrap, Bonferroni: standard experiment design.
- Pass@k estimator: HumanEval paper (Chen et al., 2021).
- Majority-vote binomial: standard probability.
- InstructGPT ~70% labeler agreement: reported methodology.
- **UNVERIFIED:** items marked [rule of thumb] are practitioner heuristics from eval-engineering writeups (suite tier sizes, 30 cases, 3-5 repeats, 20-50 calibration items, gold-item counts and the 5% seeding rate, the 80% qual bar, the 0.7/0.6 kappa bars, the 10% probe break limit), not theorems or published standards. Tune every one against your own data before it gates a release.

Where to go next

The next pack in the sequence is [Crash: Agent Systems](#). This pack measured models and kept them safe. The next one asks what happens when models act: tool use, multi-step agents, and the evals that catch agents failing.

WATCH AND GO DEEPER

Two videos on evals done right. A practitioner workshop on the eval pyramid and harnesses. A research talk on why eval signal is the bottleneck in alignment.

Strategies for LLM Evals (AI Engineer)

Taylor Smith (Red Hat) at AI Engineer covers the eval pyramid as the organizing framework. Then hands-on with GuideLLM, lm-eval-harness, and OpenAI Evals. The talk ends on evals in CI/CD. Transcript verified: the workshop version of this pack's Rule 1.

Aligning Open Language Models (Stanford CS25, Nathan Lambert)

Nathan Lambert (AI2) retells the open-alignment story from ChatGPT to DPO. Alpaca, reward models, Chatbot Arena, and why evaluation signal became the bottleneck. Transcript verified: the research backstory behind this pack's calibration and private-set rules.

GO DEEPER

lm-evaluation-harness (EleutherAI)

The standard harness: hundreds of benchmarks, one interface. The Tier 1 workhorse.

A Survey on Evaluation of Large Language Models

Chang et al.'s survey of LLM evaluation: methods, metrics, and open problems.

OWASP GenAI Top 10

The standard catalog of LLM application risks: what the red-team loop in this pack hunts for.